Assignment #3 Report - Reliable Data Transfer Protocols

Student Name: Muhammad Umar
Student ID: 23K-0023

1. Objective

This assignment implements reliable unidirectional data transfer over an unreliable channel. Three protocols are implemented and tested: rdt 3.0 (Stop-and-Wait), Go-Back-N (GBN), and Selective Repeat (SR).

2. Implementation Overview

The program is implemented in Python and organized into packet, network, sender, and receiver components. The unreliable network behavior is simulated with random packet loss, corruption, and delay.

2.1 Core Configuration

- PROTOCOL_MODE: Selects RDT3.0, GBN, or SR mode.
- TOTAL_PACKETS: Number of packets transmitted in one run.
- PACKET_SIZE: Payload size in bytes for each packet.
- WINDOW_SIZE: Sliding-window size for GBN and SR.
- TIMEOUT: Fixed retransmission timeout.
- PROB_LOSS / PROB_CORRUPT / PROB_DELAY: Error model probabilities.

2.2 Protocol Logic

rdt 3.0 (Stop-and-Wait):

- Sender transmits one packet and waits for matching ACK.
- Timer is started after transmission, timeout causes retransmission of current packet.
- Receiver uses alternating sequence numbers (0/1) and sends duplicate ACK on mismatch/corruption.

Go-Back-N (GBN):

- Sender pipelines up to WINDOW_SIZE packets.
- Receiver accepts only in-order packets and sends cumulative ACK for last in-order packet.
- On timeout, sender retransmits all packets from base to next_seq_num-1.

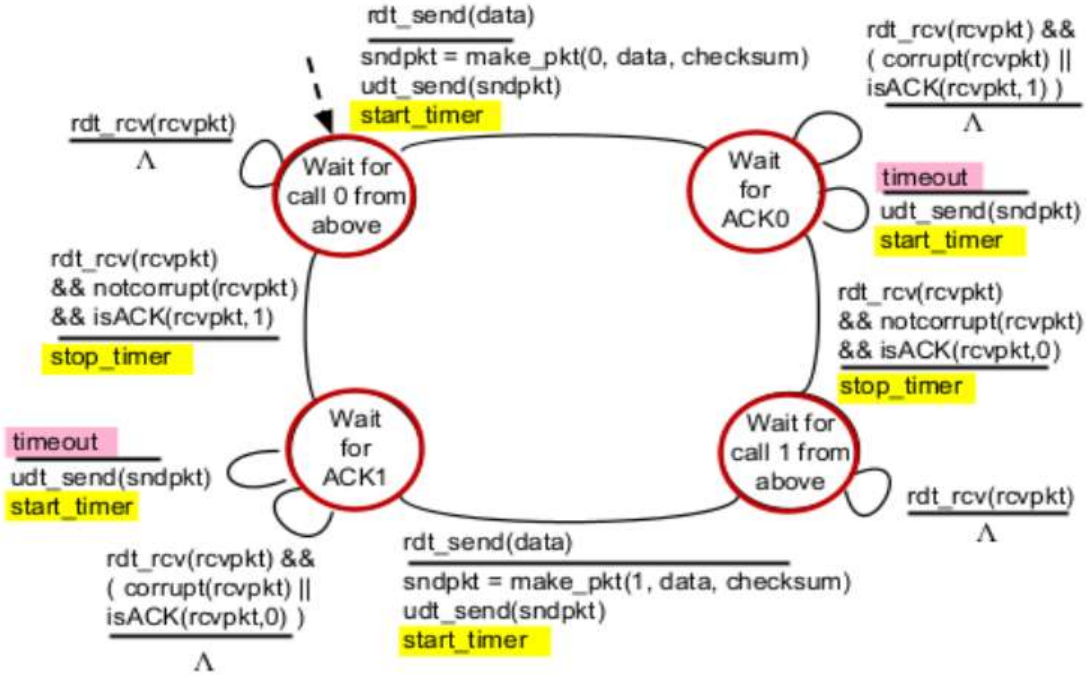
Selective Repeat (SR):

- Sender pipelines up to WINDOW_SIZE packets with per-packet timers.
- Receiver accepts and buffers out-of-order packets within the receive window.
- On timeout, sender retransmits only the timed-out packet.

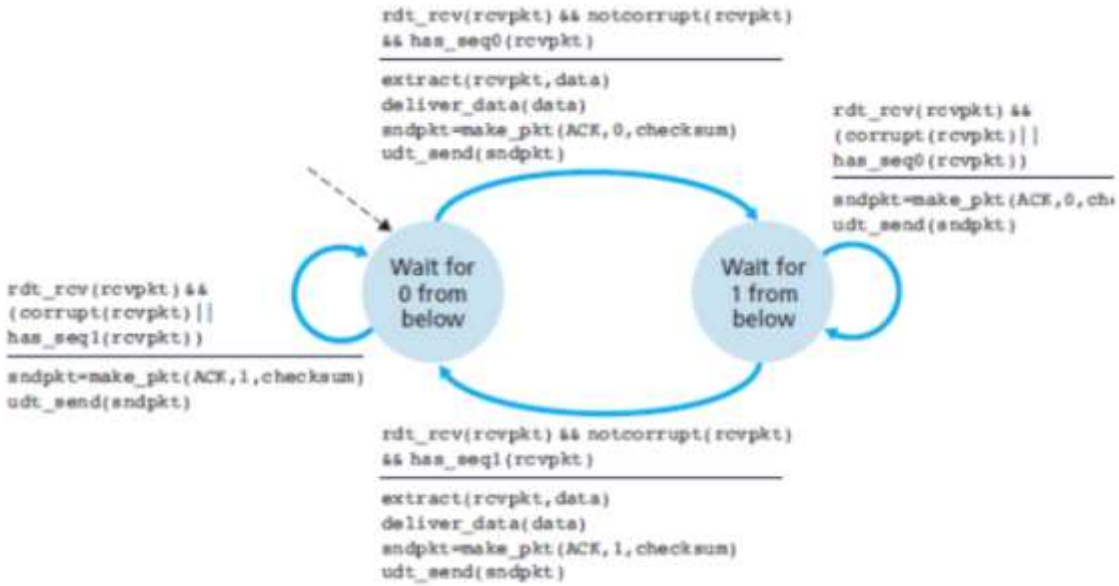
3. FSM Diagrams

FSMs used for sender and receiver behavior are shown below.

3.1 RDT 3.0

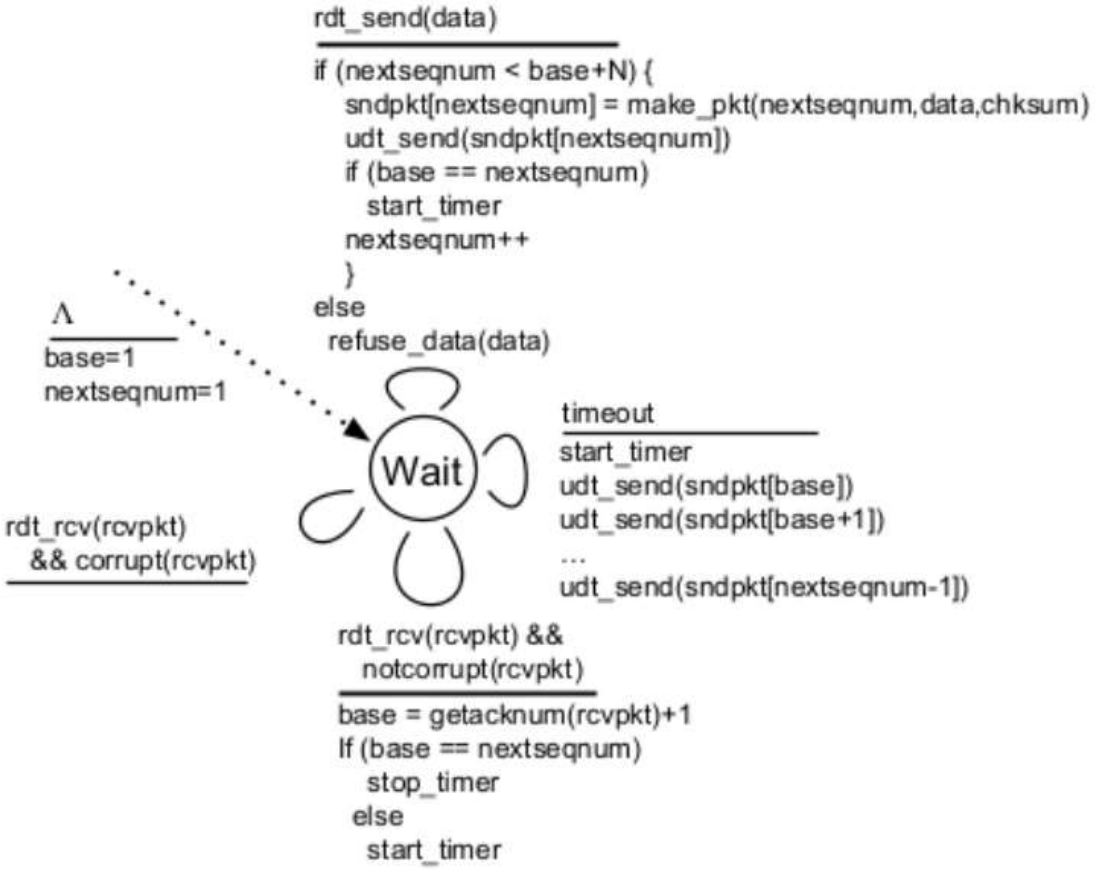


RDT 3.0 Sender FSM

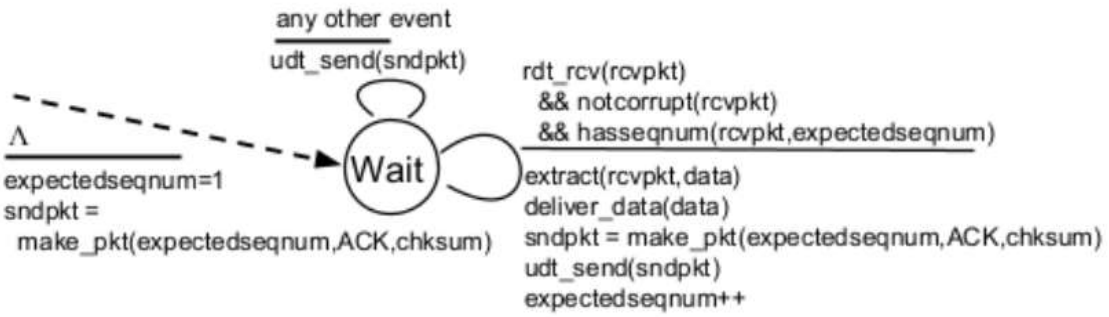


RDT 3.0 Receiver FSM

3.2 Go-Back-N



GBN Sender FSM

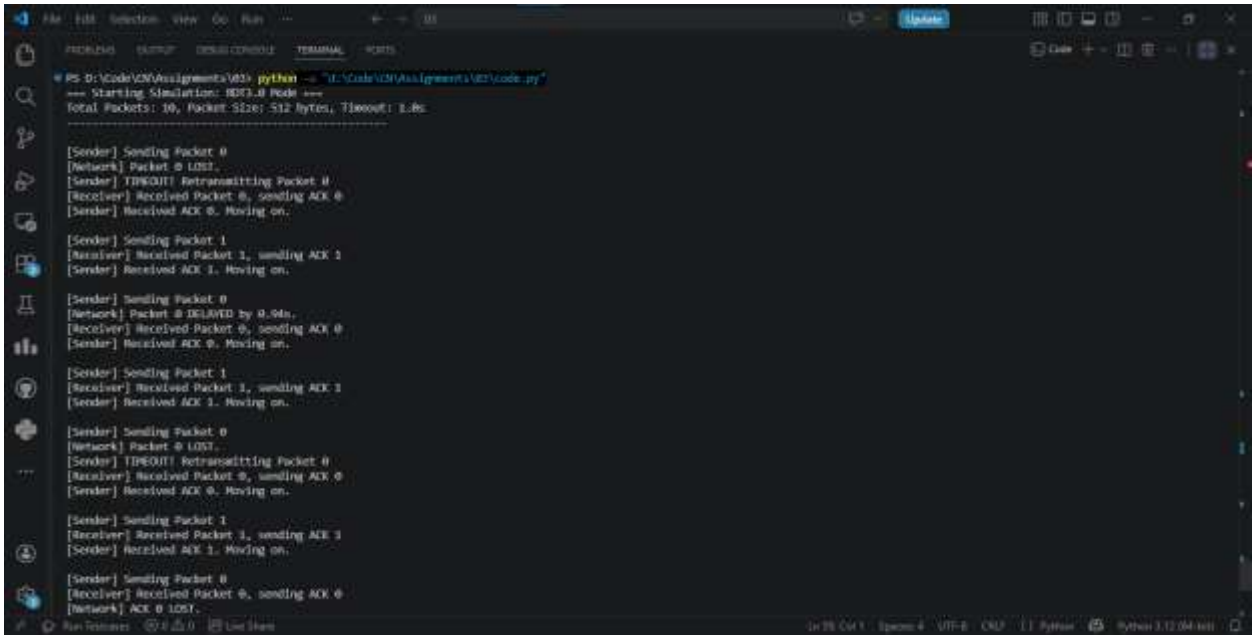


GBN Receiver FSM

4. Testing and Results

The implementation was tested under random loss, corruption, and delay. In all runs, retransmission and ACK logic ensured successful complete delivery.

4.1 RDT 3.0 Output

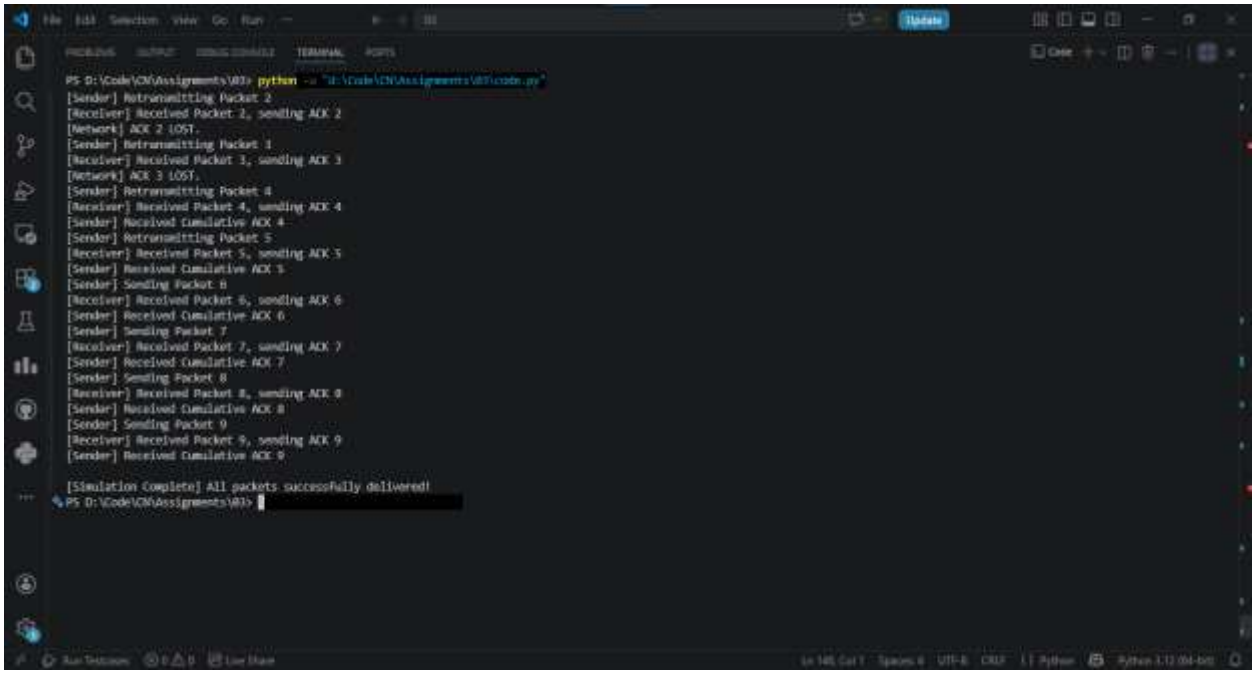


RDT 3.0 execution output (RDT1.png)

RDT 3.0 execution output (RDT2.png)

4.2 GBN Output

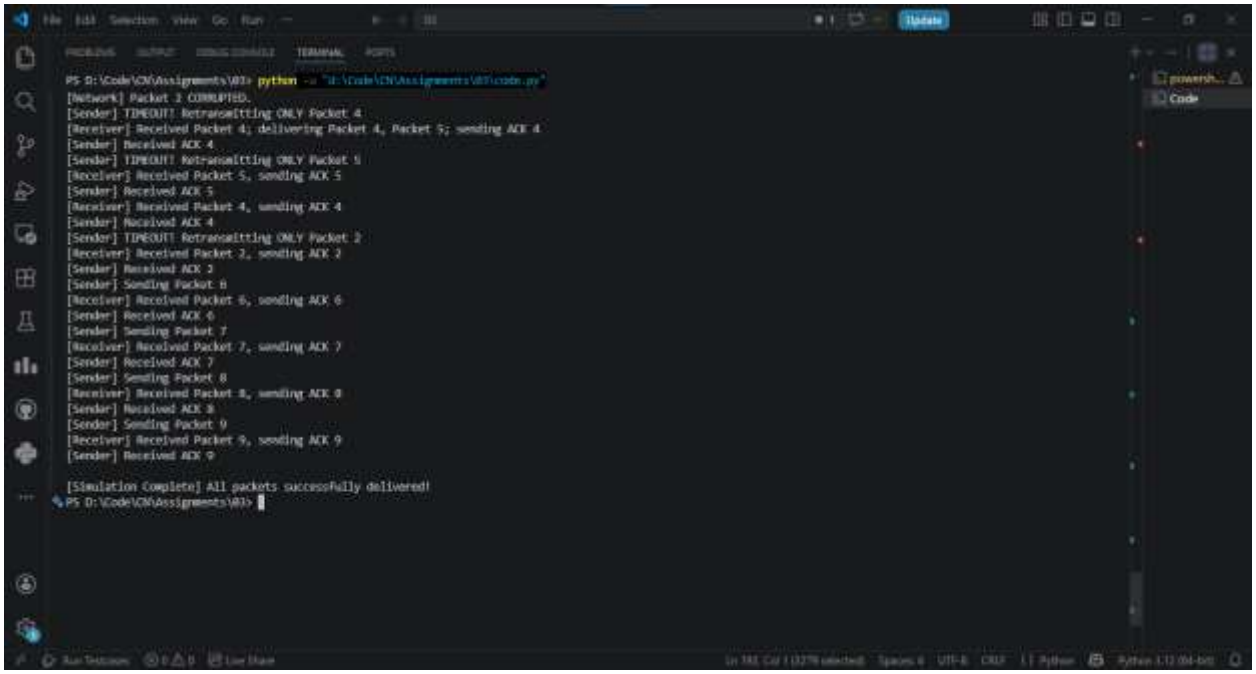
GBN execution output (GBN1.png)



GBN execution output (GBN2.png)

4.3 SR Output

SR execution output (SR1.png)



SR execution output (SR2.png)

5. Requirement Coverage Summary

Assignment Requirement	Implemented Evidence
rdt 3.0 implementation	Classes RDTReceiver and RDTSender implement stop-and-wait with alternating sequence bit.
GBN implementation	Classes GBNReceiver and GBNSender implement pipelining with cumulative ACK and retransmit-from-base.
SR implementation	Classes SRReceiver and SRSender implement buffering and per-packet timeout retransmission.
Packet loss/corruption/delay	Network class injects random loss, corruption, and delay using configurable probabilities.
FSM usage	FSM diagrams for sender and receiver are included.
Configurable packet count/size	TOTAL_PACKETS and PACKET_SIZE are configurable constants.
Fixed timeout	TIMEOUT constant controls retransmission timers for all protocols.
Testing scenarios	Screenshots for RDT 3.0, GBN, and SR runs are included, logs show loss/corruption/delay recovery.

6. How to Run

1. Open terminal in the project folder.
2. Set PROTOCOL_MODE to one of: "RDT3.0", "GBN", or "SR".
3. Optionally adjust TOTAL_PACKETS, PACKET_SIZE, WINDOW_SIZE, TIMEOUT, and error probabilities.
4. Run: python code.py
5. Observe sender/receiver logs and retransmission behavior in console output.

7. Conclusion

The implemented simulator successfully demonstrates core reliable transport mechanisms. rdt 3.0 provides baseline reliability with stop-and-wait, GBN improves throughput with cumulative ACKs, and SR further optimizes retransmissions by retransmitting only missing packets. Observed outputs confirm correct recovery from packet loss, corruption, and delayed delivery.